Unit-4

Transport Layer

- Transport Services
- Elements of Transport protocols
- Connection management
- TCP and UDP protocols.

Introduction of Transport layer

- Transport Layer is the second layer in the TCP/IP model and the fourth layer in the OSI model.
- The unit of data encapsulation in the Transport Layer is a **segment**.
- The Transport Layer is not just another layer, it is the heart of the whole protocol hierarchy.
- Its task is to provide reliable, cost-effective data transport from the source machine to destination machine, independently of the physical network or network's currently in use.
- Without the transport layer, the whole concept of layered protocols would make little sense.
- Communication is provided using a logical connection, which means that the two application layers, which can be located in different parts of the globe, assume that there is an imaginary direct connection through which they can send and receive messages.

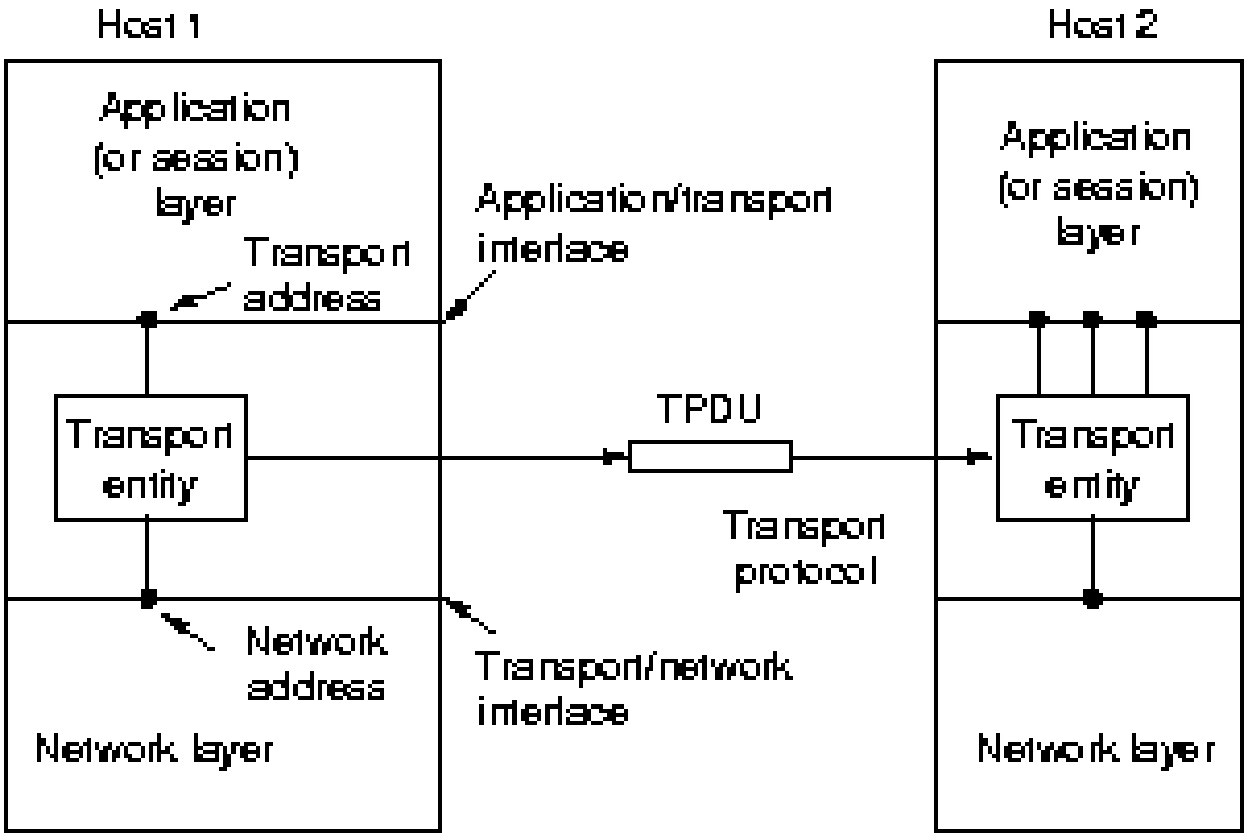
Transport layer Services

- **The main role of the transport layer is to provide the communication services directly to the application processes running on different hosts.**
- **The transport layer protocols are implemented in the end systems but not in the network routers.**
- A computer network provides more than one protocol to the network applications. For example, TCP and UDP are two transport layer protocols that provide a different set of services to the network layer.
- **All transport layer protocols provide multiplexing/demultiplexing service**.
- It also provides other services such as **reliable data transfer**, bandwidth guarantees, and delay guarantees.

- Each of the applications in the application layer has the ability to send a message by using TCP or UDP.
- The application communicates by using either of these two protocols. Both TCP and UDP will then communicate with the internet protocol in the internet layer.
- The applications can read and write to the transport layer. Therefore, we can say that communication is a two-way process.

Services Provided to the Upper Layer:

- The ultimate goal of the transport layer is to provide efficient, reliable, and cost-effective services to its users, normally processes in the application layer.
- To achieve this goal, the Transport layer makes use of the services provided by the network layer.
- **The hardware or software within the transport layer that does the work is called the Transport entity.**
- **The transport entity can be located in the OS kernel** .
- The relationship of the network, transport & application layers is shown in below figure


Host 1
Application
(or session)
layer
Transport
address
Application/transport
interface
Transport
entity
TPDU
Transport
protocol
Network
address
Transport/network
interface
Network layer
Host 2
Application
(or session)
layer
Transport
entity
Network layer

- **At the sender's side:** The transport layer receives data (message) from the Application layer and then performs Segmentation, divides the actual message into segments, adds source and destination's port numbers into the header of the segment, and transfers the message to the Network layer.
- **At the receiver's side:** The transport layer receives data from the Network layer, reassembles the segmented data, reads its header, identifies the port number, and forwards the message to the appropriate port in the Application layer.

- In network layer, there are two types of network service, connection-oriented and connectionless, there are also two types of transport service**. The connection-oriented transport service is similar to the connection-oriented network service in many ways**. The connectionless transport service is also very similar to the connectionless network service.
- In both cases, **connections have three phases: establishment, data transfer, and release.**
- Addressing and flow control are also similar in both layers.
- **The transport code runs entirely on the users machines , but the network layer mostly runs on the routers, which are operated by the carrier.**

Transport Service Primitives

- To allow users to access the transport service, the transport layer must provide some operations to application programs, that is, a transport service interface. Each transport service has its own interface.
- The transport service is similar to the network service, but there are also some important differences. The main difference is that the network service is intended to model the service offered by real networks and all. Real networks can lose packets, so the network service is generally unreliable.

- The connection-oriented transport service, in contrast, is reliable. Of course, real networks are not error-free, but that is precisely the purpose of the transport layer—to provide a reliable service on top of an unreliable network.
- A second difference between the network service and transport service is that the services are intended for. The network service is used only by the transport entities. Few users write their own transport entities, and thus few users or programs ever (meaning always/forever/still) see the bare network service.
- To get an idea of what a transport service might be like, consider the five primitives listed in below Fig.

Figure : The primitives for a simple transport service.

Primitive	Packet sent	Meaning
LISTEN	(none)	**Block until some process tries to connect**
CONNECT	CONNECTION REQ.	**Actively attempt to establish a connection**
SEND	DATA	**Send information**
RECEIVE	(none)	**Block until a DATA packet arrives**
DISCONNECT	**DISCONNECTION REQ.**	**Request a release of the connection**

- TPDU (Transport Protocol Data Unit) for messages sent from transport entity to transport entity.
- TPDU are contained in packets. Packets are contained in frames (exchanged by the data link layer).
- When a frame arrives, the data link layer processes the frame header and, if the destination address matches for local delivery, passes the contents of the frame payload field up to the network entity.
- The network entity similarly processes the packet header and then passes the contents of the packet payload up to the transport entity.
- This nesting is illustrated in below Fig.

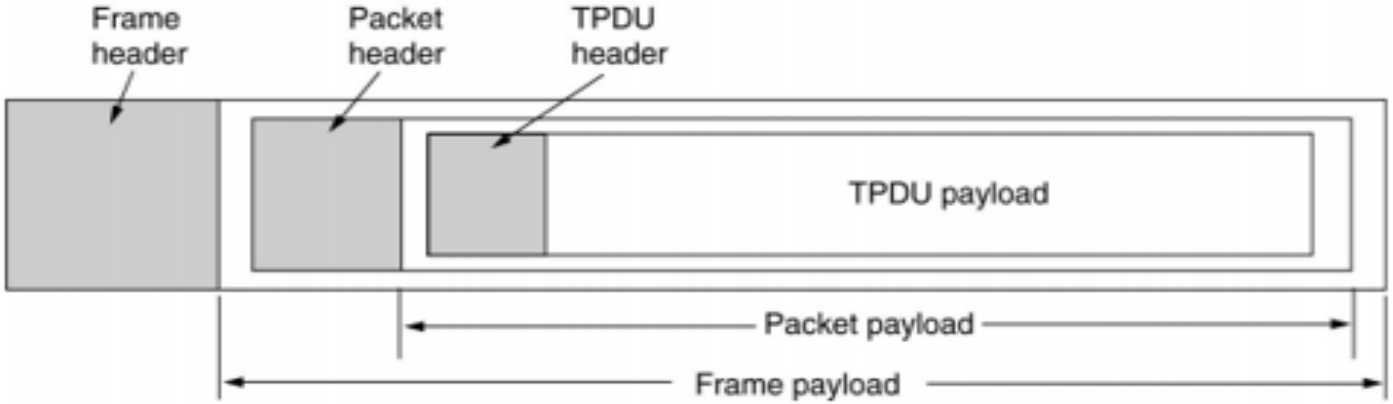
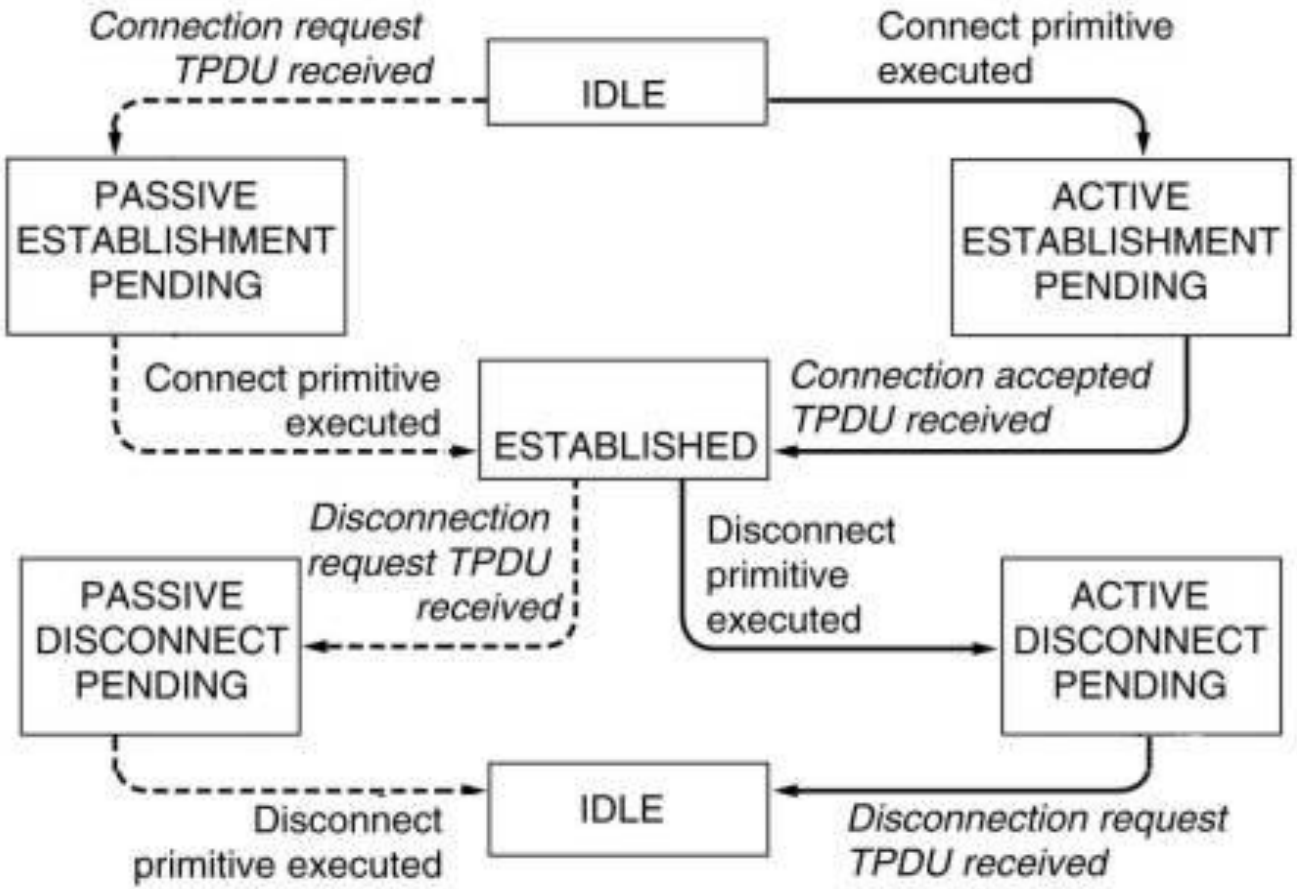


Figure 5.1 The nesting of TPDUs, packets and frames

- A state diagram for connection establishment and release for these simple primitives is given in below Fig.
- **Each transition is triggered by some event**, either a primitive executed by the local transport user or an incoming packet.
- For simplicity, we assume here that each segment is separately acknowledged. We also assume that a symmetric disconnection model is used, with the client going first. Please note that this model is quite unsophisticated.

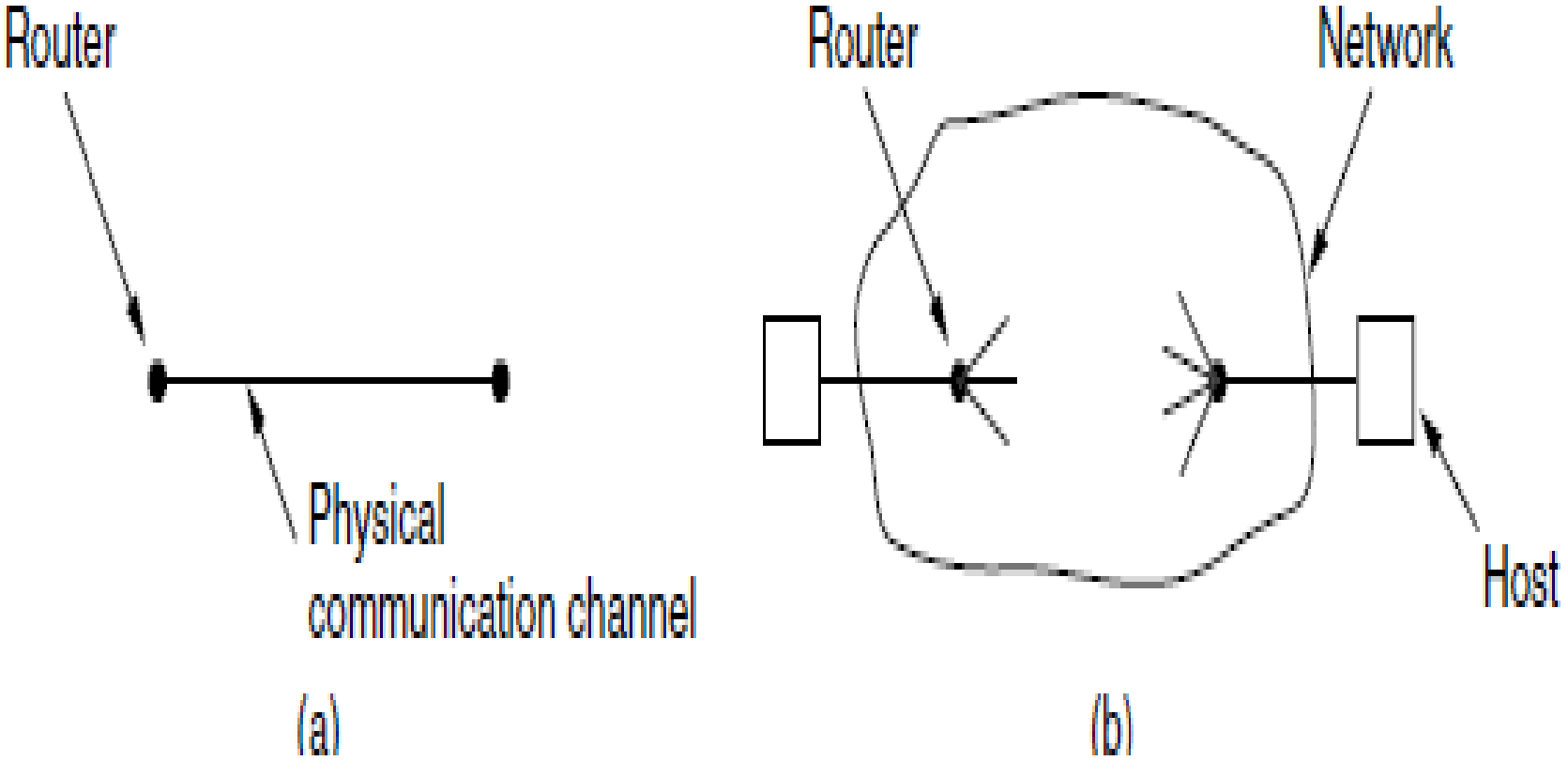
Figure: A state diagram for a simple connection management scheme. Transitions labeled in italics are caused by packet arrivals. The solid lines show the client's state sequence. The dashed lines show the server's state sequence



Elements of Transport protocols

- **The transport service is implemented by a transport protocol used between the two transport entities.**
- In some ways, transport protocols resemble the data link protocols. **Both have to deal with error control, sequencing, and flow control, among other issues.**
- However, significant differences between the two also exist. **These differences are due to major dissimilarities between the environments in which the two protocols operate**, as shown in below Fig.

Figure: Environment of the (a) data link layer (b) transport layer



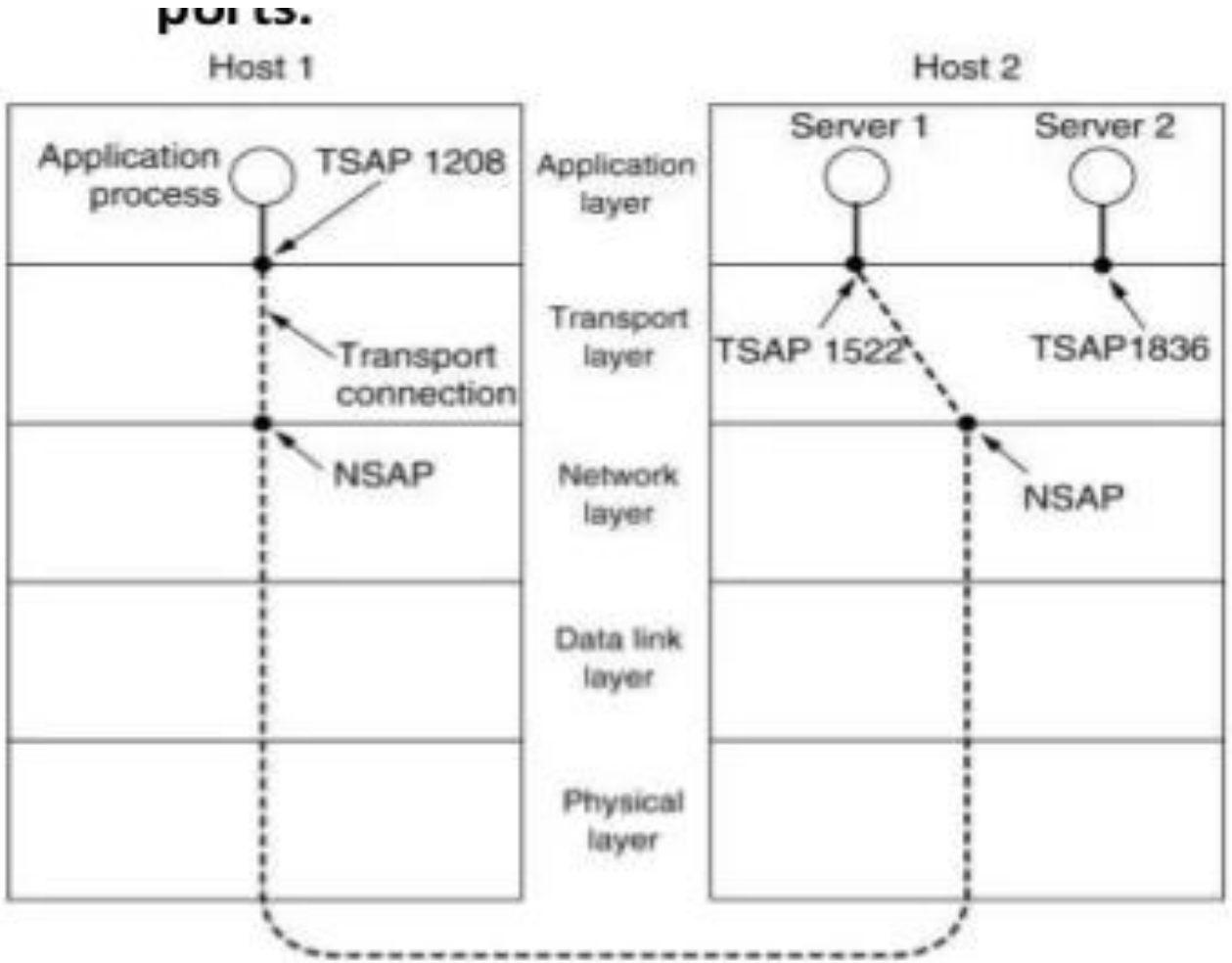
- **At the data link layer**, two routers communicate directly via a physical channel, whether wired or wireless,
- **whereas at the transport layer**, this physical channel is replaced by the entire network.
- For one thing, over point-to-point links such as **wires or optical fiber**, **it is usually not necessary for a router to specify which router it wants to talk to—each outgoing line leads directly to a particular router.**
- **In the transport layer, explicit addressing of destinations is required.**

- For another thing, **the process of establishing a connection over the wire of Fig.(a) is simple**.The other end is always there (unless it has crashed, in which case it is not there). Either way, there is not much to do.
- **Even on wireless links**, the process is not much different. Just sending a message is sufficient to have it reach all other destinations. If the message is not acknowledged due to an error, it can be resent. **In the transport layer, initial connection establishment is complicated**.

1. Addressing:

- **When an application (e.g., a user) process wishes to set up a connection to a remote application process, it must specify which one to connect to**. (Connectionless transport has the same problem: to whom should each message be sent?) The method normally used is to define **transport addresses to which processes can listen for connection requests**.
- **In the Internet, these endpoints are called ports**. We will use the generic term **TSAP (Transport Service Access Point) to mean a specific endpoint in the transport layer.** In Internet, they consists of an (IP address, local port) pair.
- The **analogous endpoints in the network layer (i.e., network layer addresses) are naturally called NSAPs (Network Service Access Points). IP addresses are examples of NSAPs.**

Figure: illustrates the relationship between the NSAPs, the TSAPs, and a transport connection.



- Application processes, both clients and servers, can attach themselves to a local TSAP to establish a connection to a remote TSAP. These connections run through NSAPs on each host, as shown in figure.
- A possible scenario for a transport connection is as follows:

1. A mail server process attaches itself to TSAP 1522 on host 2 to wait for an incoming call. A call such as our LISTEN might be used, for example.
2. An application process on host 1 wants to send an email message, so it attaches itself to TSAP 1208 and issues a CONNECT request. The request specifies TSAP 1208 on host 1 as the source and TSAP 1522 on host 2 as the destination. This action ultimately results in a transport connection being established between the application process and the server.

3. The application process sends over the mail message.
4. The mail server responds to say that it will deliver the message.
5. The transport connection is released.

2. Connection Establishment:

- Establishing a connection sounds easy, but it is actually surprisingly tricky. At first , it would seem sufficient for one transport entity to just send a CONNECTION REQUEST segment to the destination and wait for a CONNECTION ACCEPTED reply. **The problem occurs when the network can lose, delay, corrupt, and duplicate packets. This behavior causes serious complications.**
- **Imagine a network that is so congested that acknowledgements hardly ever get back in time and each packet times out and is retransmitted two or three times.** Suppose that the network uses datagram inside and that every packet follows a different route.

- **Some of the packets might get stuck in a traffic jam inside the network and take a long time to arrive**. That is, they may be delayed in the network and pop out much later, when the sender thought that they had been lost.
- The worst possible nightmare is as follows. **"A user establishes a connection with a bank, sends messages telling the bank to transfer a large amount of money to the account of a not-entirelytrustworthy person** ". Unfortunately, the packets decide to take the scenic route to the destination and go off exploring a remote corner of the network.

- **The sender then times out and sends them all again. This time the packets take the shortest route and are delivered quickly so the sender releases the connection.**
- Unfortunately, eventually the initial batch of packets finally come out of hiding and arrives at the destination in order, asking the bank to establish a new connection and transfer money (again). The bank has no way of telling that these are duplicates. It must assume that this is a second, independent transaction, and transfers the money again.
- **The root of the problem is that the delayed duplicates are thought to be new packets. We cannot prevent packets from being duplicated and delayed. But if and when this happens, the packets must be rejected as duplicates and not processed as fresh packets.**

- The problem can be attacked in various ways, none of them very satisfactory. One way is to use throwaway transport addresses. In this approach, each time a transport address is needed, a new one is generated. When a connection is released, the address is discarded and never used again. Delayed duplicate packets then never find their way to a transport process and can do no damage.

  Note: However, this approach makes it more difficult to connect with a process in the first place.

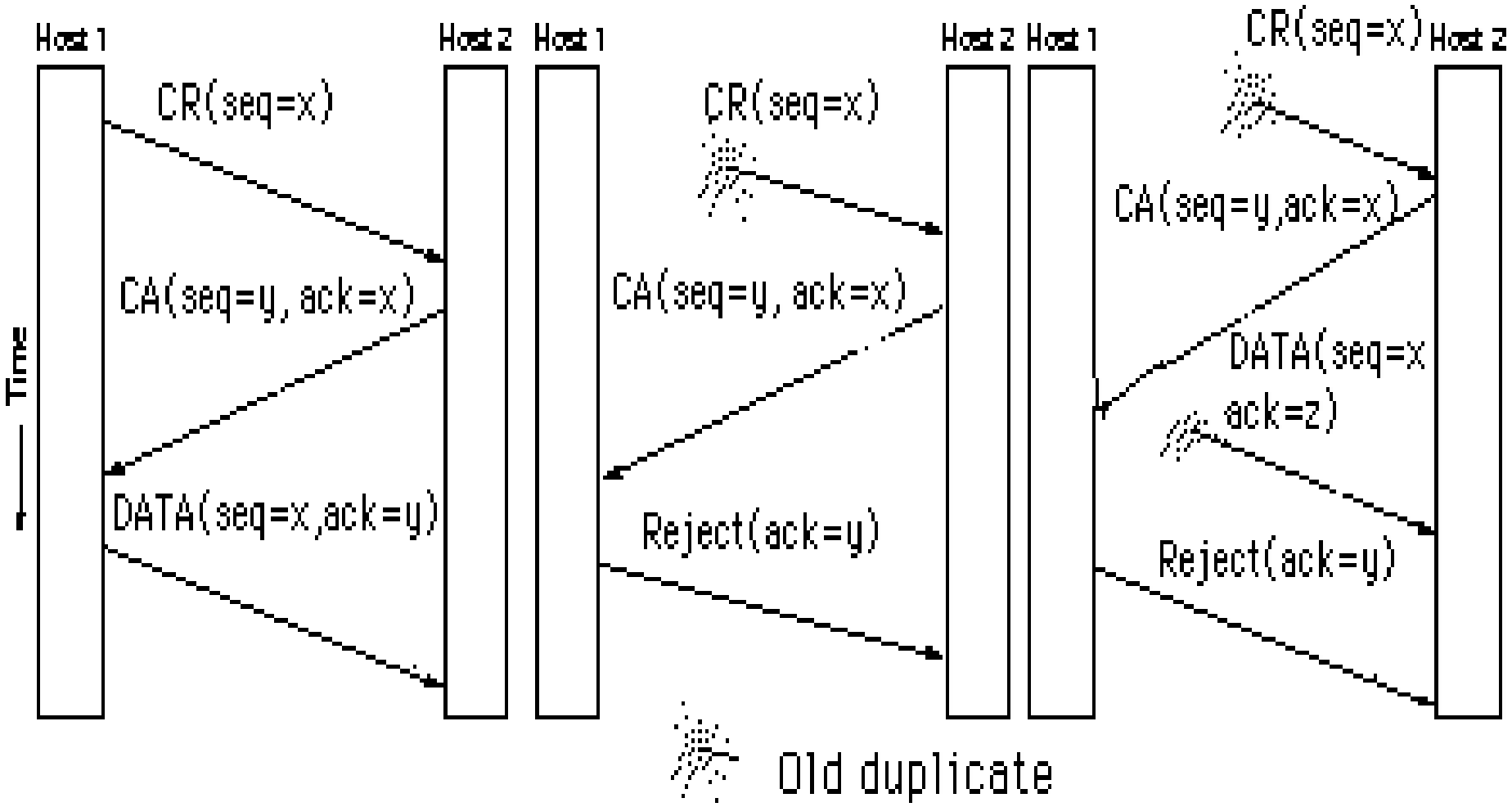
- Another possibility is to give each connection a unique identifier (i.e., a sequence number incremented for each connection established) chosen by the initiating party and put in each segment, including the one requesting the connection

- After each connection is released, each transport entity can update a table listing obsolete connections as (peer transport entity, connection identifier) pairs. Whenever a connection request comes in, it can be checked against the table to see if it belongs to a previously released connection.
- Unfortunately, this scheme has a basic flaw: it requires each transport entity to maintain a certain amount of history information indefinitely. This history must persist at both the source and destination machines. Otherwise, if a machine crashes and loses its memory, it will no longer know which connection identifiers have already been used by its peers.

- Instead, we need to take a different tack to simplify the problem. Rather than allowing packets to live forever within the network, we devise a mechanism to kill off aged packets that are still hobbling about.
- **Packet lifetime can be restricted to a known maximum using one (or more) of the following techniques:**
  1. **Restricted network design.**
  2. **Putting a hop counter in each packet.**
  3. **Time stamping each packet.**

- **TCP uses three-way handshake to establish connections in the presence of delayed duplicate control segments as shown in below figure.**

Figure 4.5: Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes Connection Request. (a) Normal operation. (b) Old duplicate connection request appearing out of nowhere. (c) Duplicate connection request and duplicate ack.



3. Releasing a connection

- Releasing a connection is easier than establishing one. **There are two styles of terminating a connection: asymmetric release and symmetric release.**
- **Asymmetric release** is the way the telephone system works: when one party hangs up, the connection is broken.
- **Symmetric release** treats the connection as two separate unidirectional connections and requires each one to be released separately.

- **Asymmetric release** is abrupt and may result in data loss. Consider the scenario of below Fig. After the connection is established, host 1 sends a segment that arrives properly at host 2. Then host 1 sends another segment.
- Unfortunately, host 2 issues a DISCONNECT before the second segment arrives. The result is that the connection is released and data are lost.
- **Symmetric release** does the job when each process has a fixed amount of data to send and clearly knows when it has sent it. In other situations, determining that all the work has been done and the connection should be terminated is not so obvious.

- One can envision a protocol in which host 1 says ‘‘I am done. Are you done too?’’ If host 2 responds: ‘‘I am done too. Goodbye, the connection can be safely released.’’

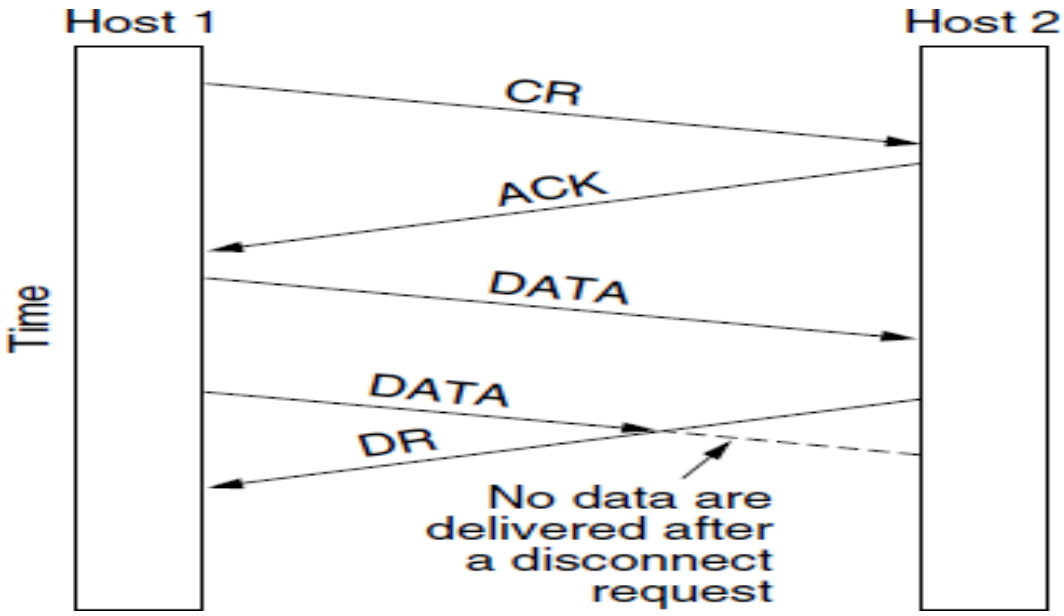
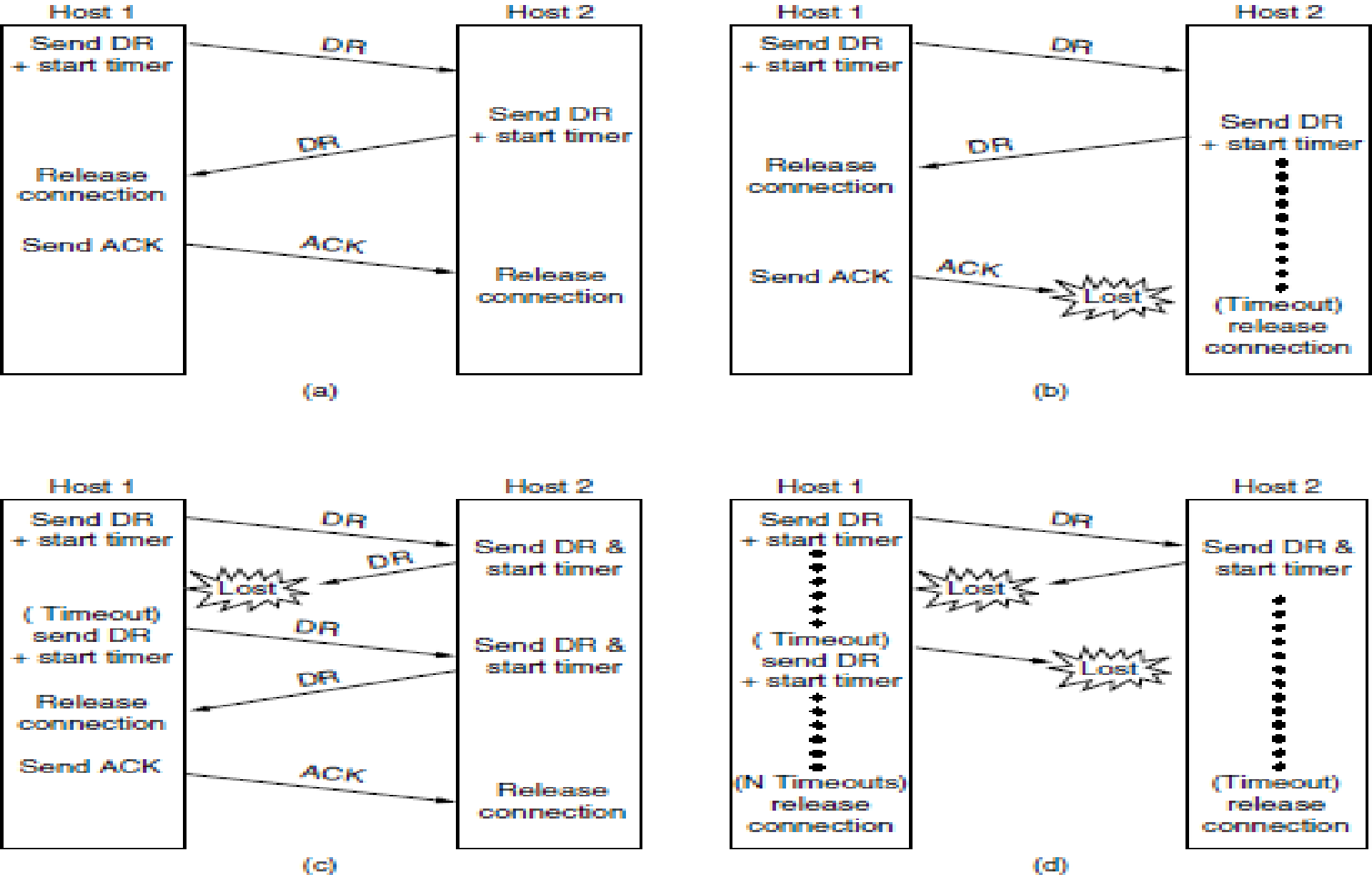


Figure : Abrupt disconnection with loss of data

- Below Figure illustrates four scenarios of releasing using a three-way handshake. While this protocol is not infallible, it is usually adequate. In Fig. (a), we see the normal case in which one of the users sends a DR (DISCONNECTION REQUEST) segment to initiate the connection release.
- When it arrives, the recipient sends back a DR segment and starts a timer, just in case its DR is lost. When this DR arrives, the original sender sends back an ACK segment and releases the connection.
- Finally, when the ACK segment arrives, the receiver also releases the connection. Releasing a connection means that the transport entity removes the information about the connection from its table of currently open connections and signals the connection's owner (the transport user) somehow.

Figure : Four protocol scenarios for releasing a connection. (a) Normal case of three-way handshake. B) Final ACK lost. (c) Response lost. (d) Response lost and subsequent DRs lost.



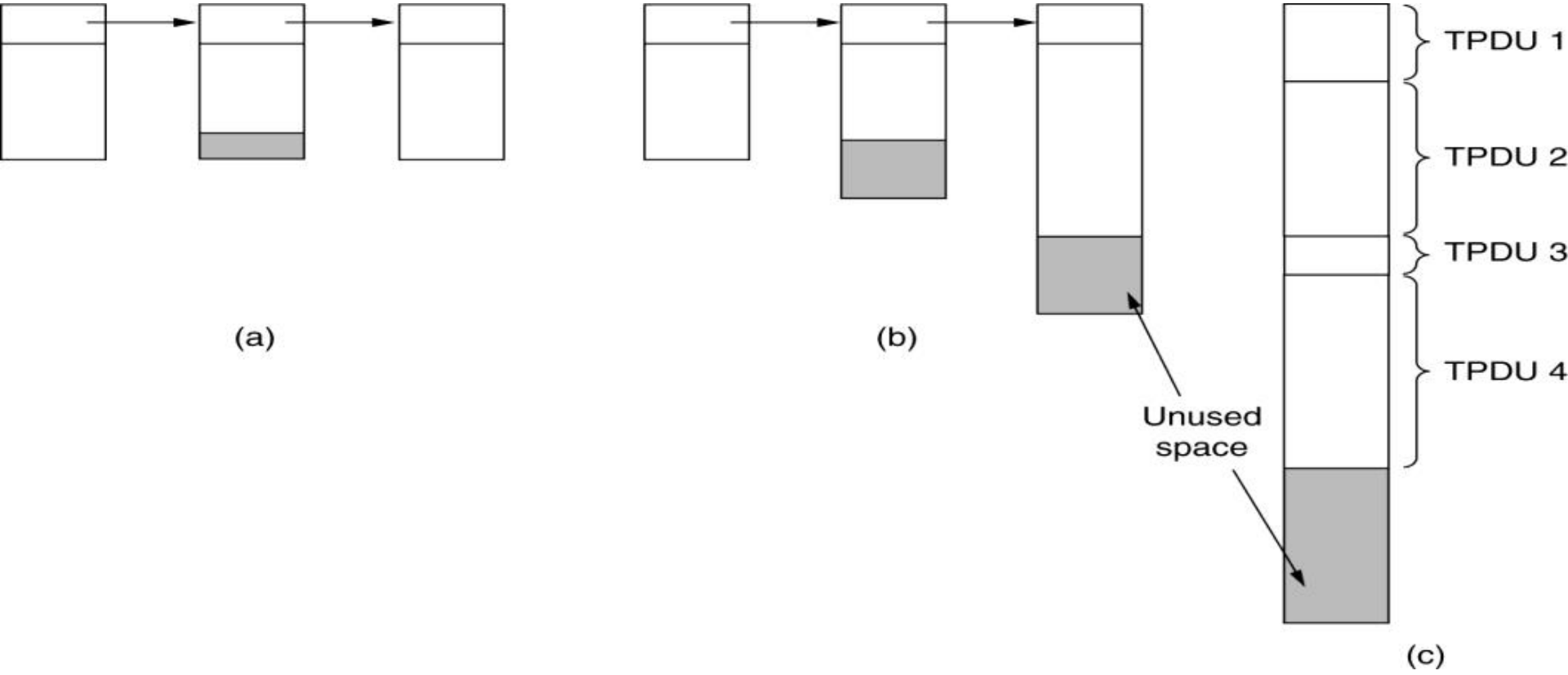
- If the final ACK segment is lost, as shown in Fig. (b), the situation is saved by the timer. When the timer expires, the connection is released anyway. Now consider the case of the second DR being lost.
- The user initiating the disconnection will not receive the expected response, will time out, and will start all over again. In Fig. (c), we see how this works, assuming that the second time no segments is lost and all segments are delivered correctly and on time.
- Our last scenario, Fig.(d), is the same as Fig.(c) except that now we assume all the repeated attempts to retransmit the DR also fail due to lost segments. After *N* retries, the sender just gives up and releases the connection. Meanwhile, the receiver times out and also exits.

4. Error control and Flow control:

- Error control is ensuring that the data is delivered with the desired level of reliability, usually that all of the data is delivered without any errors. Flow control is keeping a fast transmitter from overrunning a slow receiver.
- In some ways the flow control problem in the transport layer is the same as in the data link layer, but in other ways it is different.
- The basic similarity is that in both layers a sliding window or other scheme is needed on each connection to keep a fast transmitter from overrunning a slow receiver.
- The main difference is that a router usually has relatively few lines, whereas a host may have numerous connections.

- This difference makes it impractical to implement the data link buffering strategy in the transport layer.
- If the network service is unreliable, the sender must buffer all TPDUs sent, just as in the data link layer.
- With reliable network service, other trade-offs become possible.
- **In particular, if the sender knows that the receiver always has buffer space, it need not retain copies of the TPDUs it sends.**
- If the receiver cannot guarantee that every incoming TPDU will be accepted, the sender will have to buffer anyway.
- **In the latter case, the sender cannot trust the network layer's acknowledgement, because the acknowledgement means only that the TPDU arrived, not that it was accepted.**

Fig. 4.11: (a) Chained fixed-size buffers. (b) Chained variable-sized buffers. C) One large circular buffer per connection.



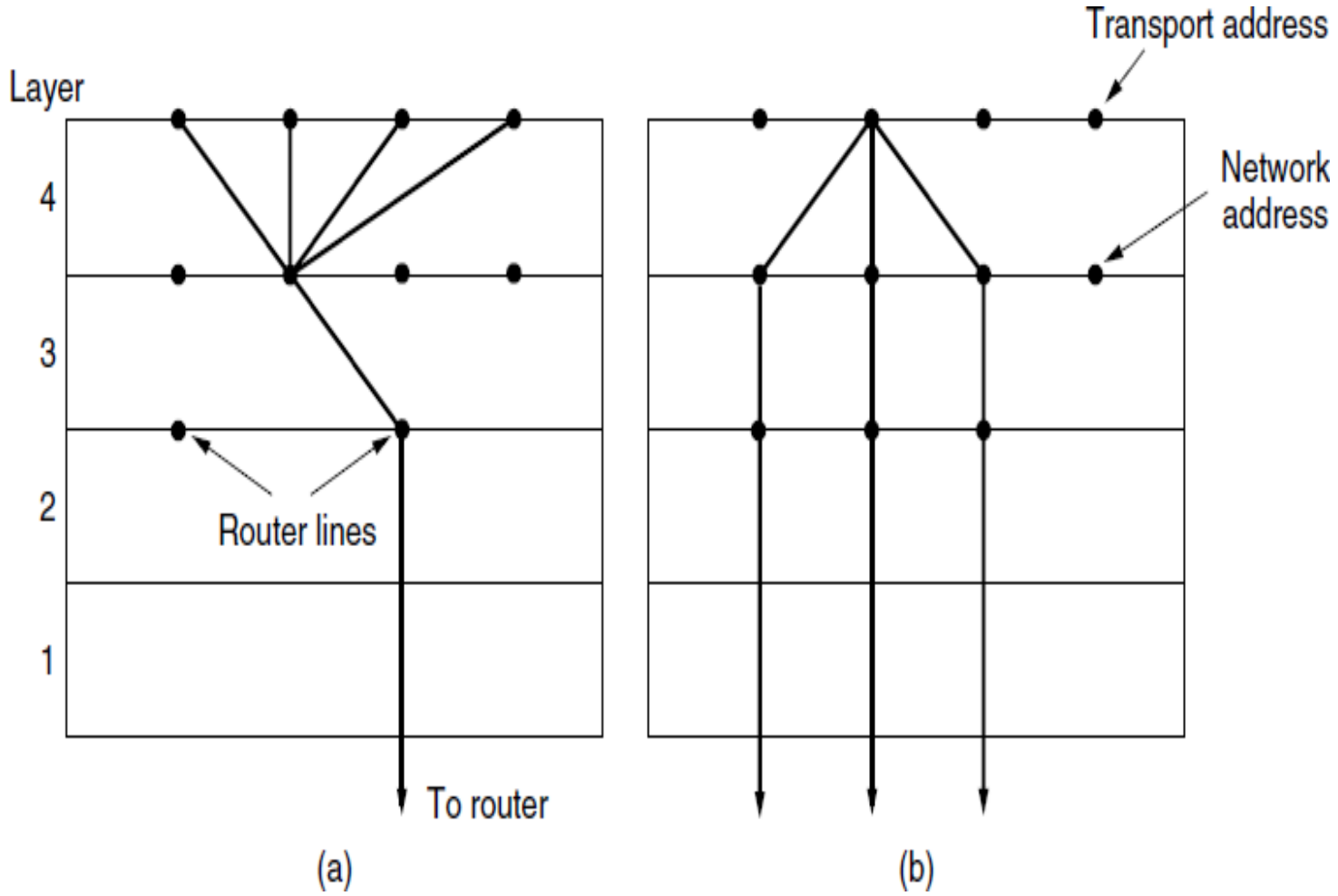
- Even if the receiver has agreed to do the buffering, there still remains the question of the buffer size.
- If most TPDUs are nearly the same size, it is natural to organize the buffers as a pool of identically- sized buffers, with one TPDU per buffer, as in Fig.(a).
- If there is wide variation in TPDU size, a pool of fixed-sized buffers presents problems.
- If the buffer size is chosen equal to the largest possible TPDU, space will be wasted whenever a short TPDU arrives.
- If the buffer size is chosen less than the maximum TPDU size, multiple buffers will be needed for long TPDUs, with the attendant complexity.

- **Another approach to the buffer size problem is to use variable-sized buffers, as in Fig. (b).**
- The advantage here is better memory utilization, at the price of more complicated buffer management.
- **A third possibility is to dedicate a single large circular buffer per connection, as in Fig. (c).**
- This system also makes good use of memory, provided that all connections are heavily loaded, but is poor if some connections are lightly loaded.

5. Multiplexing:

- Multiplexing, or sharing several conversations over connections, virtual circuits, and physical links plays a role in several layers of the network architecture. In the transport layer, the need for multiplexing can arise in a number of ways. For example, if only one network address is available on a host, all transport connections on that machine have to use it.
- When a segment comes in, some way is needed to tell which process to give it to. This situation, called **multiplexing**, is shown in Fig. (a). In this figure, four distinct transport connections all use the same network connection (e.g., IP address) to the remote host.

Figure : (a) Multiplexing (b) Inverse Multiplexing



- Multiplexing can also be useful in the transport layer for another reason. Suppose, for example, that a host has multiple network paths that it can use. If a user needs more bandwidth or more reliability than one of the network paths can provide, a way out is to have a connection that distributes the traffic among multiple network paths on a round-robin basis, as indicated in Fig.(b).

6. Crash Recovery:

- If hosts and routers are subject to crashes or connections are long-lived (e.g., large software or media downloads), recovery from these crashes becomes an issue.
- If the transport entity is entirely within the hosts, recovery from network and router crashes is straightforward. The transport entities expect lost segments all the time and know how to cope with them by using retransmissions.
- A more trouble some problem is how to recover from host crashes. In particular, it may be desirable for clients to be able to continue working when servers crash and quickly reboot.

The Internet Transport Protocols(TCP, UDP)

- The Internet has two main protocols in the transport layer, a connectionless protocol (UDP)and a connection-oriented (TCP). The protocols complement each other.

I. UDP (User Datagram Protocol):

- it is a part of the TCP/IP protocol, so it is a standard protocol over the internet. The UDP protocol allows the computer applications to send the messages in the form of datagrams from one machine to another machine over the Internet Protocol (IP) network.
- The UDP is an alternative communication protocol to the TCP protocol (transmission control protocol). Like TCP.
- UDP provides a set of rules that governs how the data should be exchanged over the internet. **The UDP works by encapsulating the data into the packet and providing its own header information to the packet. Then, this UDP packet is encapsulated to the IP packet and sent off to its destination.**

- Both the TCP and UDP protocols send the data over the internet protocol network, so it is also known as TCP/IP and UDP/IP.
- There are many differences between these two protocols. **UDP enables the process to process communication**, whereas the **TCP provides host to host communication**. Since **UDP sends the messages in the form of datagrams**, it is considered the best-effort mode of communication.
- TCP sends the individual packets, so it is a reliable transport medium. Another difference is that the TCP is a connection-oriented protocol whereas, **the UDP is a connectionless protocol as it does not require any virtual circuit to transfer the data.**

Features of UDP protocol:

- It is considered an unreliable protocol, and it is based on best-effort delivery services. UDP provides no acknowledgment mechanism, which means that the receiver does not send the acknowledgment for the received packet, and the sender also does not wait for the acknowledgment for the packet that it has sent.
- **Connectionless:** The UDP is a connectionless protocol as it does not create a virtual path to transfer the data. It does not use the virtual path, so packets are sent in different paths between the sender and the receiver, which leads to the loss of packets or received out of order.

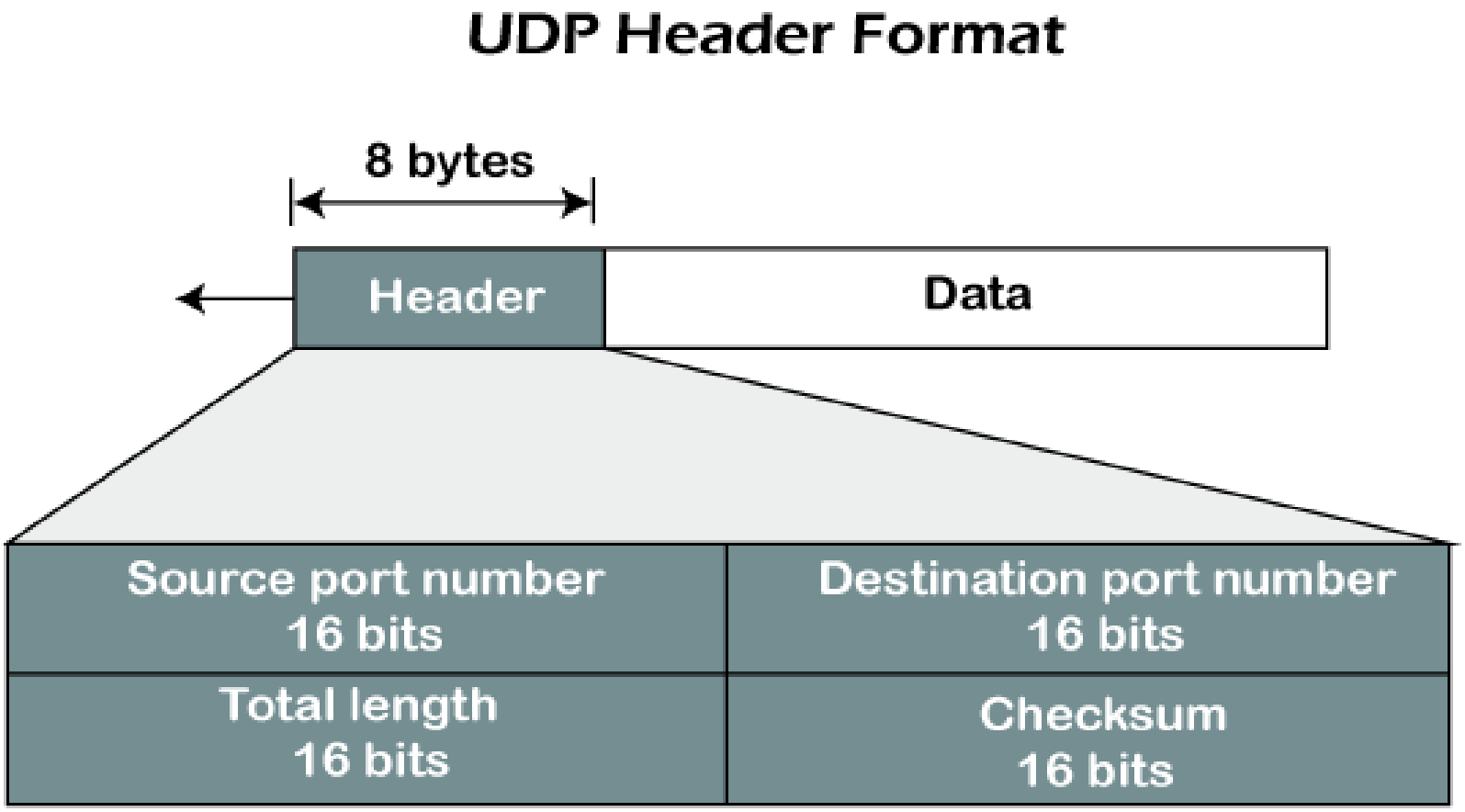
- **Ordered delivery of data is not guaranteed:** In the case of UDP, the datagrams are sent in some order will be received in the same order is not guaranteed as the datagrams are not numbered.
- **Ports:** The UDP protocol uses different port numbers so that the data can be sent to the correct destination. The port numbers are defined between 0 and 1023.
- **Faster transmission**: UDP enables faster transmission as it is a connectionless protocol, i.e., no virtual path is required to transfer the data. But there is a chance that the individual packet is lost, which affects the transmission quality. On the other hand, if the packet is lost in TCP connection, that packet will be resent, so it guarantees the delivery of the data packets.

- **Acknowledgment mechanism**: The UDP does have any acknowledgment mechanism, i.e., there is no handshaking between the UDP sender and UDP receiver. If the message is sent in TCP, then the receiver acknowledges that I am ready, then the sender sends the data. In the case of TCP, the handshaking occurs between the sender and the receiver, whereas in UDP, there is no handshaking between the sender and the receiver.
- **Segments are handled independently:** Each UDP segment is handled individually of others as each segment takes different path to reach the destination. The UDP segments can be lost or delivered out of order to reach the destination as there is no connection setup between the sender and the receiver.

Why do we require the UDP protocol?

- As we know that the UDP is an unreliable protocol, but we still require a UDP protocol in some cases. The UDP is deployed where the packets require a large amount of bandwidth along with the actual data. For example, in video streaming, acknowledging thousands of packets is troublesome and wastes a lot of bandwidth. In the case of video streaming, the loss of some packets couldn't create a problem, and it can also be ignored.

UDP Header Format



- In UDP, the header size is 8 bytes, and the packet size is upto 65,535 bytes. But this packet size is not possible as the data needs to be encapsulated in the IP datagram, and an IP packet, the header size can be 20 bytes.
- therefore, the maximum of UDP would be 65,535 minus 20. The size of the data that the UDP packet can carry would be 65,535 minus 28 as 8 bytes for the header of the UDP packet and 20 bytes for IP header.

The UDP header contains four fields:

- **Source port number:** It is 16-bit information that identifies which port is going t send the packet.
- **Destination port number:** It identifies which port is going to accept the information. It is 16-bit information which is used to identify application-level service on the destination machine.
- **Length:** It is 16-bit field that specifies the entire length of the UDP packet that includes the header also. The minimum value would be 8-byte as the size of the header is 8 bytes.
- **Checksum:** It is a 16-bits field, and it is an optional field. This checksum field checks whether the information is accurate or not as there is the possibility that the information can be corrupted while transmission. In UDP, the checksum field is applied to the entire packet, i.e., header as well as data part whereas, in IP, the checksum field is applied to only the header field.

Disadvantages:

- It provides an unreliable connection delivery service. It does not provide any services of IP except that it provides process-to-process communication.
- The UDP message can be lost, delayed, duplicated, or can be out of order.
- It does not provide a reliable transport delivery service. It does not provide any acknowledgment or flow control mechanism. However, it does provide error control to some extent.

Advantages

- It produces a minimal number of overheads.

Applications of UDP:

- Used for simple request-response communication when the size of data is less and hence there is lesser concern about flow and error control.
- It is a suitable protocol for multicasting as UDP supports packet switching.
- UDP is used for some routing update protocols like RIP(Routing Information Protocol).
- Normally used for real-time applications which can not tolerate uneven delays between sections of a received message.

TCP (Transmission Control Protocol)

- UDP is a simple protocol and it has some very important uses, such as client server interactions and multimedia, but for most Internet applications, reliable, sequenced delivery is needed. UDP cannot provide this, so another protocol is required. It is called TCP.
- It is a transport layer protocol that facilitates the transmission of packets from source to destination. **It is a connection-oriented protocol that means it establishes the connection prior to the communication that occurs between the computing devices in a network.**
- **This protocol is used with an IP protocol, so together, they are referred to as a TCP/IP.**

- The main functionality of the TCP is to take the data from the application layer. Then it divides the data into a several packets, provides numbering to these packets, and finally transmits these packets to the destination. The TCP, on the other side, will reassemble the packets and transmits them to the application layer.
- The transport layer has a critical role in providing end-to-end communication to the directly application processes. It creates 65,000 ports so that the multiple applications can be accessed at the same time. It takes the data from the upper layer, and it divides the data into smaller packets and then transmits them to the network layer.

Features of TCP protocol

The following are the features of a TCP protocol:

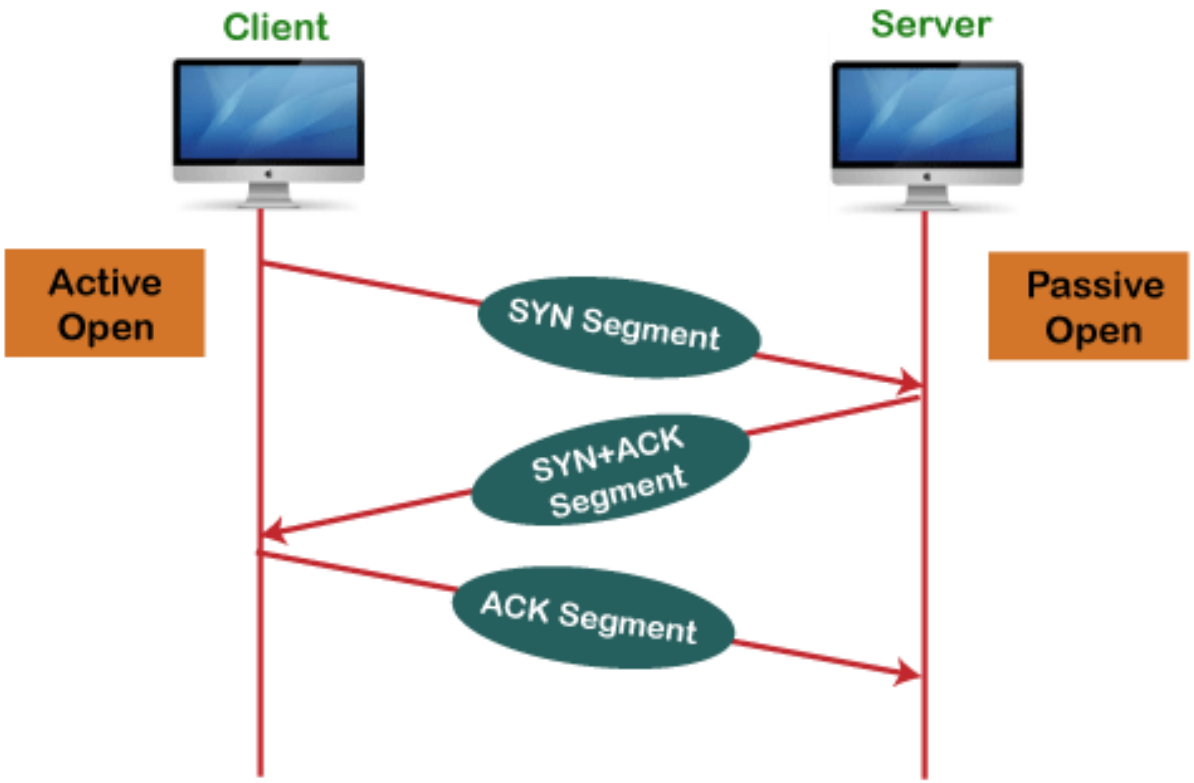
- **Reliable**: TCP is a reliable protocol as it follows the flow and error control mechanism. It also supports the acknowledgment mechanism, which checks the state and sound arrival of the data. In the acknowledgment mechanism, the receiver sends either positive or negative acknowledgment to the sender so that the sender can get to know whether the data packet has been received or needs to resend.

- **Order of the data is maintained**: This protocol ensures that the data reaches the intended receiver in the same order in which it is sent. It orders and numbers each segment so that the TCP layer on the destination side can reassemble them based on their ordering.
- **Connection-oriented**: It is a connection-oriented service that means the data exchange occurs only after the connection establishment. When the data transfer is completed, then the connection will get terminated.
- **Stream-oriented :**TCP is a stream-oriented protocol as it allows the sender to send the data in the form of a stream of bytes and also allows the receiver to accept the data in the form of a stream of bytes. TCP creates an environment in which both the sender and receiver are connected by a virtual circuit. This virtual circuit carries the stream of bytes across the internet.

Working of TCP

- **In TCP, the connection is established by using three-way handshaking.** The client sends the segment with its sequence number. **The server**, in return, sends its segment with its **own sequence number** as well as the **acknowledgement sequence**, which is one more than the client sequence number. When the client receives the acknowledgment of its segment, then it sends the acknowledgment to the server. In this way, the connection is established between the client and the server.

Working of the TCP protocol



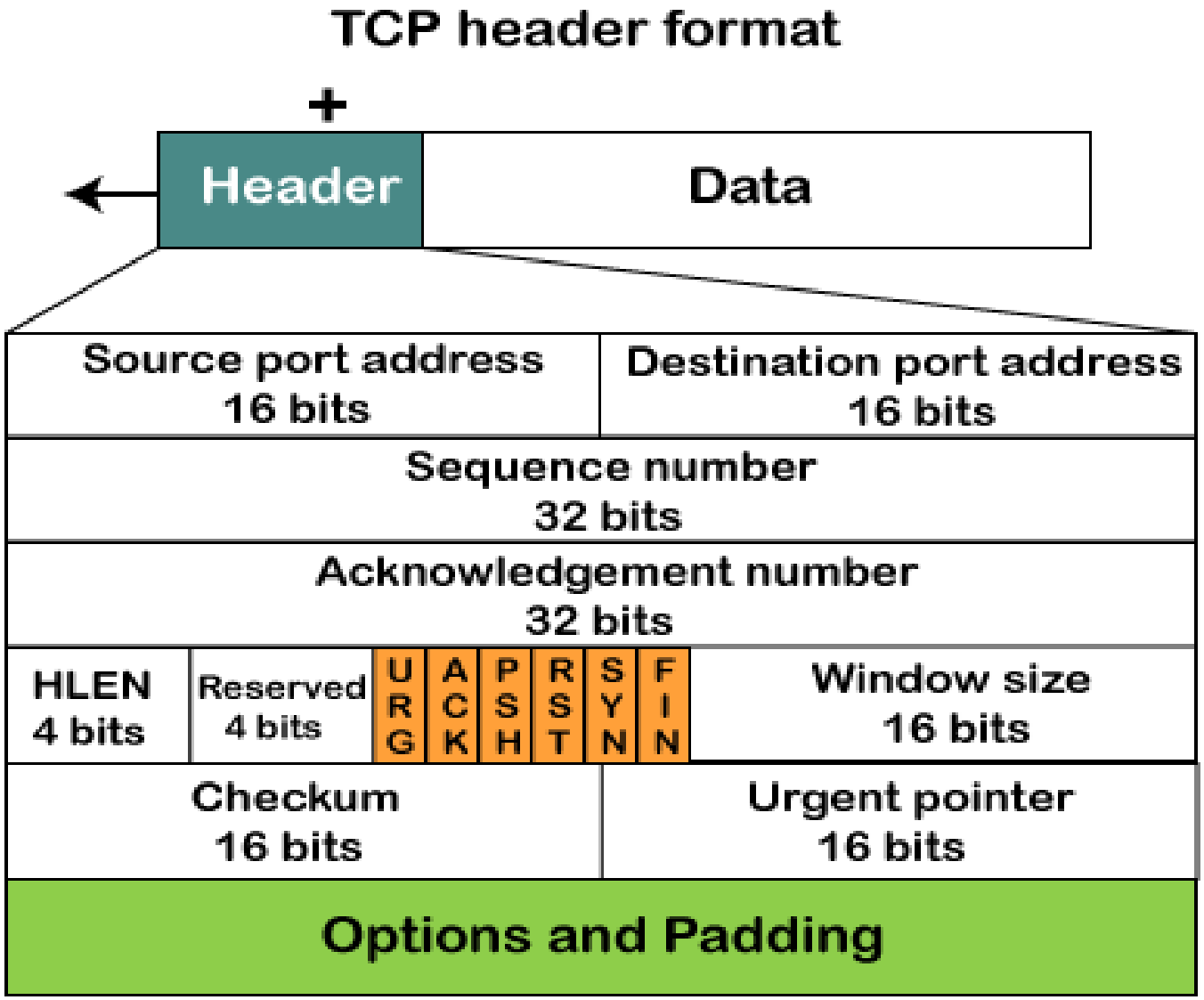
Advantages of TCP

- It provides a connection-oriented reliable service, which means that it guarantees the delivery of data packets. If the data packet is lost across the network, then the TCP will resend the lost packets.
- It provides a flow control mechanism using a sliding window protocol.
- It provides error detection by using checksum and error control by using Go Back or ARP protocol.
- It eliminates the congestion by using a **network congestion avoidance algorithm that includes various schemes such as additive increase/multiplicative decrease (AIMD), slow start, and congestion window.**

Disadvantage of TCP

- It increases a large amount of overhead as each segment gets its own TCP header, so fragmentation by the router increases the overhead.
- TCP is made for Wide Area Networks, thus its size can become an issue for small networks with low resources.
- TCP runs several layers so it can slow down the speed of the network.

TCP Header Format



- **Source port:** It defines the port of the application, which is sending the data. So, this field contains the source port address, which is 16 bits.
- Destination port: It defines the port of the application on the receiving side. So, this field contains the destination port address, which is 16 bits.
- Sequence number: This field contains the sequence number of data bytes in a particular session.
- Acknowledgment number: When the ACK flag is set, then this contains the next sequence number of the data byte and works as an acknowledgment for the previous data received. For example, if the receiver receives the segment number 'x', then it responds 'x+1' as an acknowledgment number.

- HLEN: It specifies the length of the header indicated by the 4-byte words in the header. The size of the header lies between 20 and 60 bytes. Therefore, the value of this field would lie between 5 and 15.
- Reserved: It is a 4-bit field reserved for future use, and by default, all are set to zero.
- Window size: It is a 16-bit field. It contains the size of data that the receiver can accept. This field is used for the flow control between the sender and receiver and also determines the amount of buffer allocated by the receiver for a segment. The value of this field is determined by the receiver.

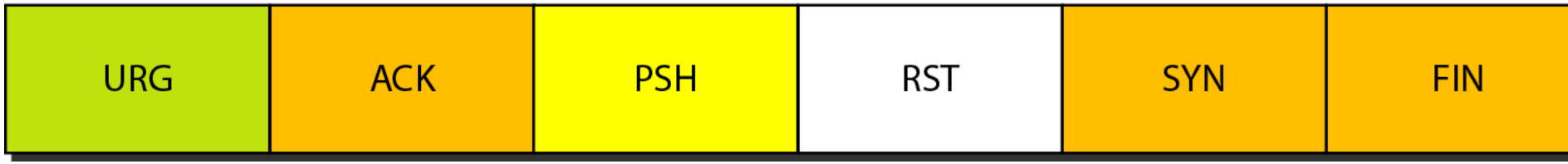
- **Flags**
- **There are six control bits or flags:**
- **URG:** It represents an urgent pointer. If it is set, then the data is processed urgently.
- **ACK:** If the ACK is set to 0, then it means that the data packet does not contain an acknowledgment.
- **PSH:** If this field is set, then it requests the receiving device to push the data to the receiving application without buffering it.
- **RST:** If it is set, then it requests to restart a connection.
- **SYN:** It is used to establish a connection between the hosts.
- **FIN:** It is used to release a connection, and no further data exchange will happen.
-

- **Checksum:** It is a 16-bit field. This field is optional in UDP, but in the case of TCP/IP, this field is mandatory.
- **Urgent pointer:** It is a pointer that points to the urgent data byte if the URG flag is set to 1. It defines a value that will be added to the sequence number to get the sequence number of the last urgent byte.
- **Options:** It provides additional options. The optional field is represented in 32-bits. If this field contains the data less than 32-bit, then padding is required to obtain the remaining bits.

- These bits enable flow control, connection establishment and termination, connection abortion, and the mode of data transfer in TCP.

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push

RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection



A TCP Connection

- TCP is connection-oriented. A connection-oriented transport protocol
- establishes a virtual path between the source and destination. All the segments belonging to a message are then sent over this virtual path. Using a single virtual pathway for the entire message facilitates the acknowledgment process as well as retransmission of damaged or lost frames.
- In TCP, connection-oriented transmission requires 2 phases:
    - 1. connection establishment,
    - 2. connection termination.

1. TCP connection establishment(3 way handshaking):

Step 1: SYN

- SYN is a segment sent by the client to the server. It acts as a **connection request** between the client and server. It informs the server that the client wants to establish a connection.
- The client sends the first segment, a SYN segment, in which only the SYN flag is set. NOTE:A SYN segment cannot carry data, but it consumes one sequence number.

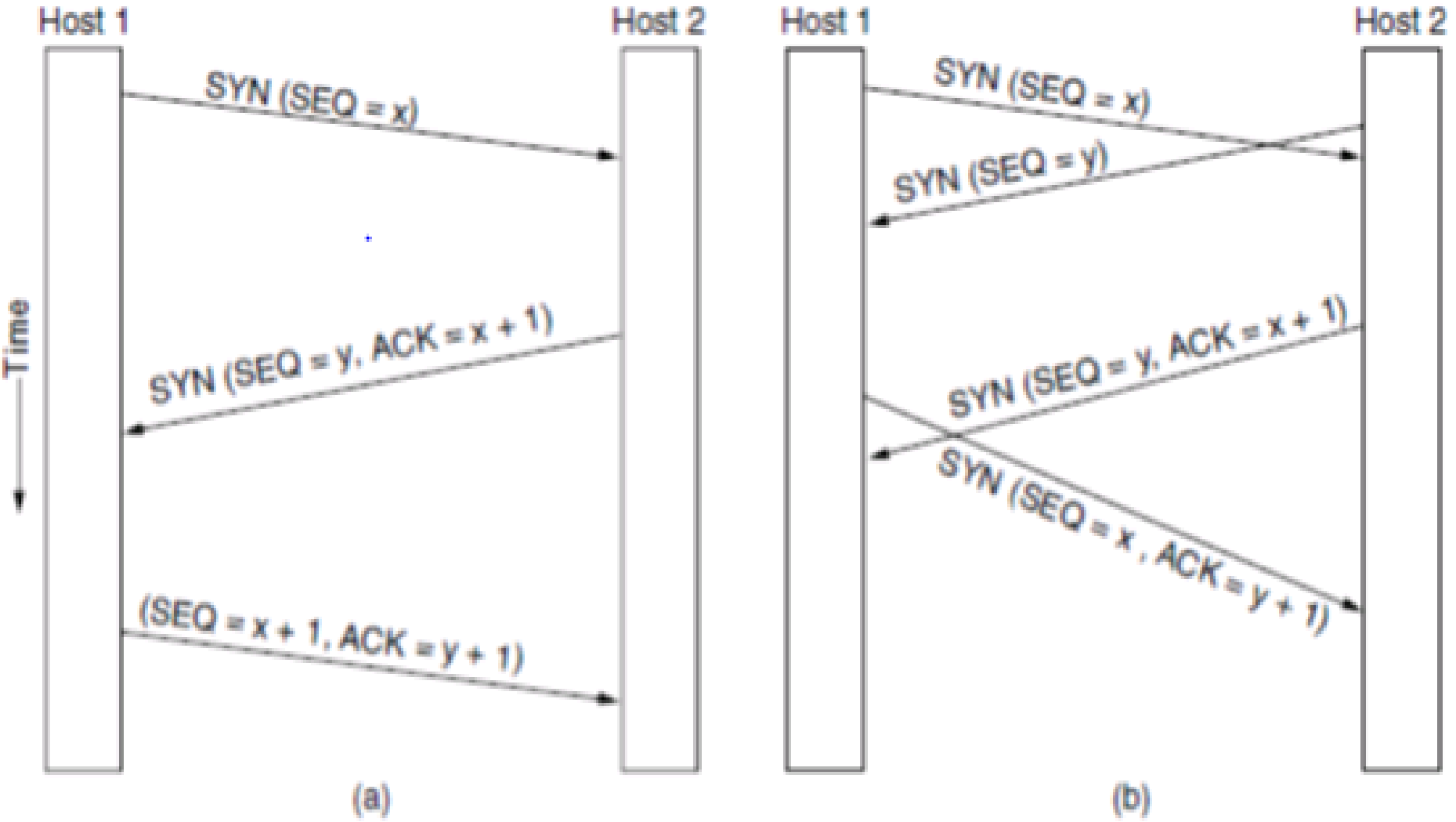
Step 2: SYN-ACK

- It is an SYN-ACK segment or an SYN + ACK segment sent by the server. The ACK segment informs the client that the server has received the connection request and it is ready to build the connection. The SYN segment informs

NOTE: A SYN+ACK segment cannot carry data, but does consume one sequence number.

Step 3: ACK

- ACK (Acknowledgment) is the last step before establishing a successful TCP connection between the client and server. The ACK segment is sent by the client as the response of the received ACK and SN from the server. It results in the establishment of a reliable data connection.
- After these three steps, the client and server are ready for the data communication process. **TCP connection and termination are full-duplex, which means that the data can travel in both the directions simultaneously.**



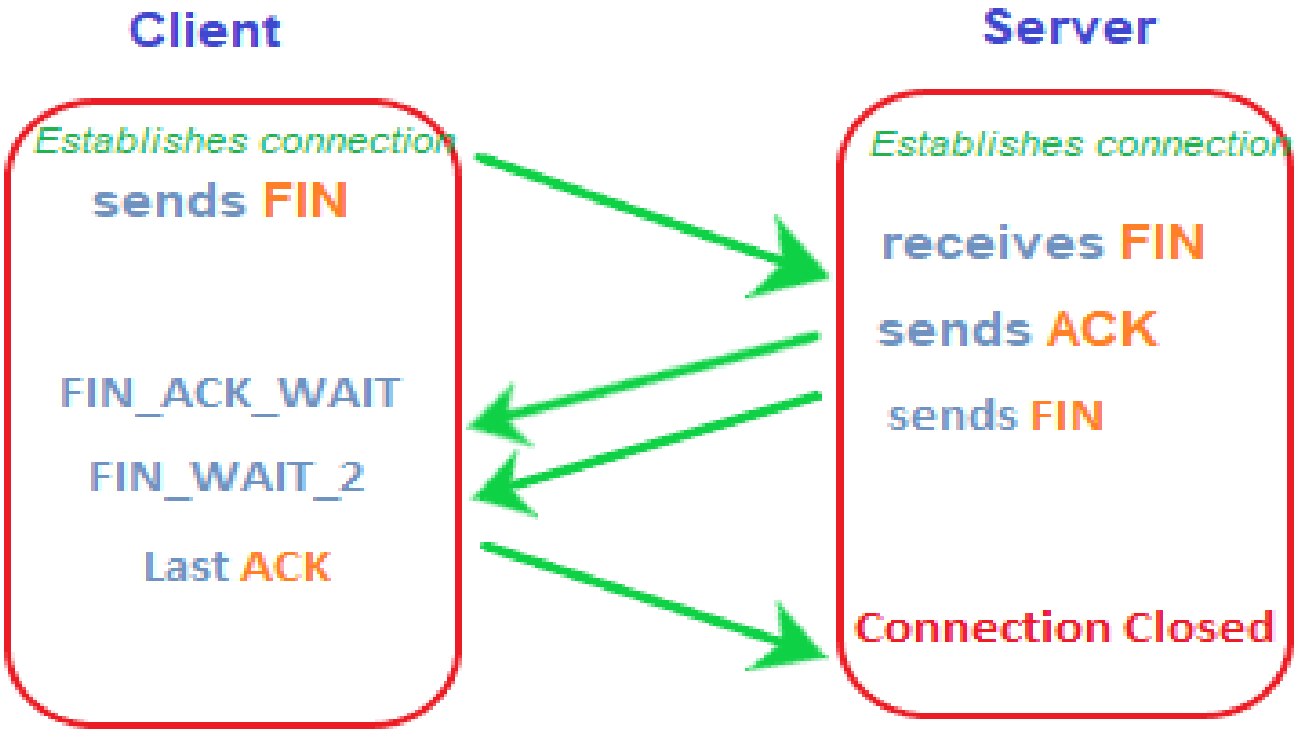
a) TCP connection establishment in the normal case. (b) Simultaneous connection establishment on both sides.

- **In the event that two hosts simultaneously attempt to establish a connection between the same two sockets**, the sequence of events is as illustrated in Fig.(b). The result of these events is that just one connection is established, not two, because connections are identified by their end points. If the first setup results in a connection identified by (x, y) and the second one does too, only one table entry is made, namely, for (x, y).

2. connection termination (A 4-way handshake)

- Any device establishes a connection before proceeding with the termination. TCP requires 3-way handshake to establish a connection between the client and server before sending the data.
- Similarly, to terminate or stop the data transmiss ion, it requires a 4-way handshake. The segments required for TCP termination are similar to the segments to build a TCP connection (ACK and SYN) except the FIN segment. The FIN segment specifies a termination request sent by one device to the other.

- The client is the data transmitter and the server is a receiver in a data transmission process between the sender and receiver. Consider the below TCP termination diagram that shows the exchange of segments between the client and server.
- The diagram of a successful TCP termination showing the f



Step 1: FIN

- FIN refers to the **termination request** sent by the client to the server. The first FIN termination request is sent by the client to the server. It depicts the start of the termination process between the client and server.

Step 2: FIN_ACK_WAIT

- The client waits for the ACK of the FIN termination request from the server. It is a **waiting state** for the client.

Step 3: ACK

- The server sends the ACK (Acknowledgement) segment when it receives the FIN termination request. It depicts that the server is ready to close and terminate the connection.

Step 4: FIN _WAIT_2

- The client waits for the FIN segment from the server. It is a type of approved signal sent by the server that shows that the server is ready to terminate the connection.

Step 5: FIN

- The FIN segment is now sent by the server to the client. It is a confirmation signal that the server sends to the client. It depicts the successful approval for the termination.

Step 6: ACK

- The client now sends the ACK (Acknowledgement) segment to the server that it has received the FIN signal, which is a signal from the server to terminate the connection. As soon as the server receives the ACK segment, it terminates the connection.

- Flow Control or TCP Sliding Window
- TCP Congestion Control
- Tcp timer management